

CI/CD et Automatisation

Compréhension des Pipelines CI/CD

CI/CD (Intégration Continue et Déploiement Continu) a été présenté précédemment comme une pratique clé du **DevOps** pour automatiser le cycle de vie du développement logiciel.

Il est maintenant temps de voir comment cela fonctionne en pratique — à travers des exemples concrets. Cela vous aidera à comprendre pourquoi le **CI/CD** est si important et ce qui peut mal tourner sans lui.

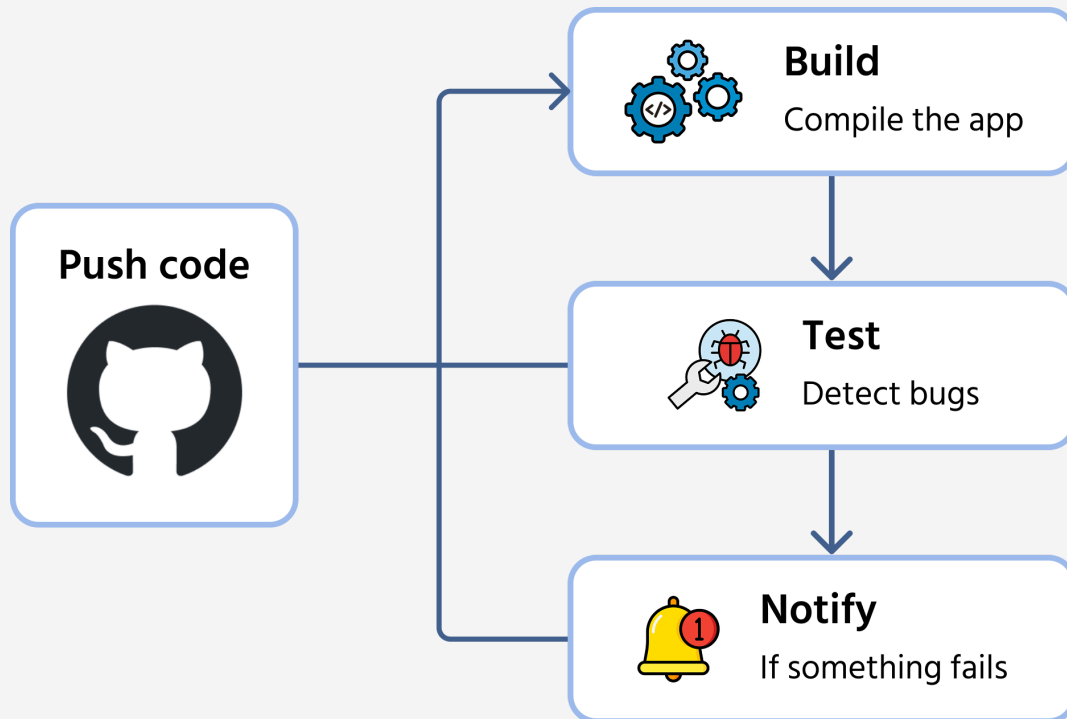
Intégration Continue (CI)

L'**Intégration Continue** permet de tester et d'intégrer automatiquement le code à chaque modification. Le code est généralement poussé plusieurs fois par jour, et à chaque mise à jour, le processus CI vérifie si tout fonctionne toujours correctement.

Exemple :

Vous poussez une modification de code dans un dépôt partagé (comme **GitHub**). Juste après, le **système CI** :

- Compile l'application pour vérifier qu'elle se construit correctement ;
- Exécute des tests automatisés pour détecter les bogues ;
- Envoie une notification en cas d'échec.



Cela permet d'identifier les problèmes rapidement — avant qu'ils n'affectent le reste de l'équipe ou le produit.

Sans CI :

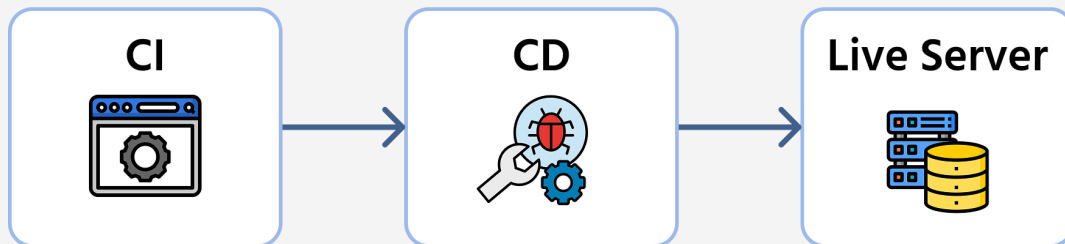
Si votre équipe n'utilise pas **l'intégration continue**, toutes les modifications risquent d'être fusionnées seulement une fois par semaine (voire moins souvent). C'est à ce moment-là que des bogues cachés apparaissent — et il devient beaucoup plus difficile de déterminer quel changement a causé le problème. Corriger ces problèmes peut prendre des heures, voire des jours, ralentissant ainsi l'ensemble du projet.

Déploiement Continu (CD)

Une fois que votre code a passé les tests dans le **CI**, le **Déploiement Continu** le livre automatiquement en production — l'environnement réel utilisé par les utilisateurs. Aucune étape manuelle, aucun délai d'attente.

Exemple :

Vous ajoutez une nouvelle fonctionnalité à une application web. Après que le système de **CI** a confirmé son bon fonctionnement :



- Le **pipeline CD** envoie automatiquement la mise à jour sur le serveur de production, où les utilisateurs peuvent y accéder ;
- La mise à jour est déployée de manière à ne pas interrompre les utilisateurs — l'application continue de fonctionner pendant l'ajout de la nouvelle version ;
- En cas de problème, le système revient rapidement à la dernière version fonctionnelle, évitant ainsi tout désagrément pour les utilisateurs.

Cela rend la publication de nouvelles fonctionnalités fluide, rapide et sécurisée.

Sans CD :

Sans **CD**, vous devez déployer les mises à jour manuellement — par exemple via un script ou en téléchargeant les fichiers vous-même. Ce processus est lent et risqué. Une petite erreur (comme l'oubli d'un fichier ou l'exécution d'une mauvaise commande) peut entraîner une panne du système ou exposer les utilisateurs à des fonctionnalités défectueuses.

Pourquoi le CI/CD est important

Si vous débutez avec le **DevOps**, considérez le **CI/CD** comme votre assistant d'automatisation personnel :

- **Vous écrivez du code** → il est testé automatiquement ;
- **Vous terminez une fonctionnalité** → elle est déployée automatiquement.

Cela permet de gagner du temps, de réduire les erreurs et de se concentrer sur le développement des fonctionnalités — sans se soucier des risques de casse ou des modalités de mise en production.

Maîtriser le **CI/CD** dès le début offre un avantage considérable pour livrer des logiciels plus rapidement et de manière plus fiable.

Outils utilisés dans les pipelines CI/CD

Catégorie	Outils populaires
Gestion de versions	Git, GitHub, GitLab, Bitbucket
Serveurs CI/CD	Jenkins, GitHub Actions, CircleCI, TravisCI
Conteneurisation	Docker, Kubernetes
Déploiement cloud	AWS CodePipeline, Google Cloud Build, Azure DevOps
Supervision	Prometheus, ELK Stack, Datadog
Infrastructure as Code (IaC)	Terraform, Ansible

Dans ce cours, vous utiliserez certains des outils DevOps les plus répandus, notamment **Git, GitHub, Jenkins, GitHub Actions, Docker, Kubernetes, Terraform, Ansible, Prometheus** et la **ELK Stack**. Ces outils constituent la base des workflows **CI/CD** modernes.

Tout au long de ce cours, vous découvrirez comment chacun de ces outils s'intègre dans le pipeline **CI/CD** — et comment les utiliser ensemble pour construire des workflows **DevOps** robustes et automatisés.

Introduction à Jenkins

Si vous débutez dans l'exploration du **DevOps** et du **CI/CD**, l'un des premiers outils que vous rencontrerez est **Jenkins**. Il s'agit d'un élément central du développement logiciel moderne qui permet d'automatiser les tâches répétitives et de rendre le processus de déploiement plus rapide et plus fiable.

Qu'est-ce que Jenkins ?



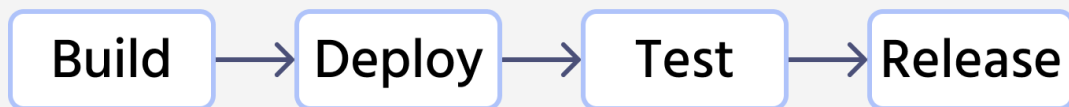
Définition

Jenkins est un outil d'automatisation gratuit et open source conçu pour rationaliser le processus de développement logiciel. Il est utilisé pour construire, tester et déployer automatiquement des applications — ce qui est particulièrement utile pour les équipes effectuant des modifications fréquentes du code et ayant besoin de résultats rapides et cohérents.

Considérez **Jenkins** comme un assistant automatisé. Il surveille votre code et exécute des tâches importantes pour vous, sans intervention manuelle.

Chaque fois qu'un nouveau code est envoyé dans un dépôt, **Jenkins** peut immédiatement vérifier s'il fonctionne.

Fonctionnement de Jenkins dans la pratique



Imaginez que vous avez écrit du code et que vous l'avez poussé vers un système de gestion de versions comme **GitHub**. **Jenkins** détecte ce changement et déclenche une séquence d'étapes — appelée pipeline. Ce pipeline peut construire le projet, exécuter des tests automatisés, vérifier la présence de bugs et, si tout est conforme, préparer et même déployer une nouvelle version de l'application.

Tout cela se déroule en arrière-plan, automatiquement. Il n'est pas nécessaire d'exécuter manuellement les tests ou de téléverser des fichiers — **Jenkins** s'en charge. Cela accélère le processus de développement, réduit les risques d'erreurs et permet à l'équipe de se concentrer davantage sur la création de nouvelles fonctionnalités.

Pourquoi Jenkins est-il important ?

Le développement logiciel moderne évolue rapidement. Plus vous pouvez automatiser de tâches, moins vous risquez d'oublier des étapes ou de commettre des erreurs. **Jenkins** simplifie l'ensemble du processus de livraison, le rendant plus fluide, plus cohérent et plus facile à gérer — en particulier lors du travail en équipe ou lors de mises à jour fréquentes.

Pour toute personne débutant en **DevOps**, **Jenkins** constitue un excellent point de départ. Vous découvrirez comment créer un flux de travail où le code est testé et déployé automatiquement, sans avoir à répéter les mêmes étapes encore et encore.

Configuration d'un Pipeline Jenkins



Définition

Une **Pipeline** dans **Jenkins** est une séquence d'étapes automatisées qui s'exécutent à chaque modification du code.

L'avantage principal est qu'une **pipeline** est enregistrée directement dans le projet (dans un fichier appelé **Jenkinsfile**). Cela permet à toute l'équipe de visualiser le fonctionnement du processus de build et de le mettre à jour si nécessaire.

Les **pipelines** offrent également de la flexibilité :

- Pour de petites modifications, **Jenkins** peut exécuter uniquement des vérifications rapides ;
- Pour des mises à jour plus importantes, il peut lancer davantage de tests ou même déployer l'application.

Ainsi, une **pipeline** ne se limite pas à "**build-test-deploy**" ; il s'agit d'un ensemble de règles personnalisables permettant de garantir la sécurité de chaque modification avant qu'elle n'atteigne les utilisateurs.



Remarque

L'objectif principal d'une pipeline est de permettre l'**Intégration Continue** et la **Livraison Continue (CI/CD)**, afin que les modifications puissent être livrées rapidement et en toute sécurité aux utilisateurs.

Étapes pour créer un pipeline Jenkins

Création d'un pipeline Jenkins : une série structurée d'étapes automatisées permettant de faire passer le code du développement au déploiement.

Résumé rapide de la vidéo :

1. Se connecter à **Jenkins** ;
2. Créer un nouvel élément : cliquer sur **Nouvel élément**, saisir le nom du projet et choisir **Pipeline** comme type de projet ;
3. Configurer le pipeline :
4. Dans la section Pipeline, choisir **Pipeline Script** (manuel) ou **Pipeline Script** depuis SCM pour récupérer le **Jenkinsfile** depuis le dépôt.
5. Se connecter au dépôt **GitHub** :
6. Sous Définition, sélectionner **Pipeline Script** depuis **SCM** ;
7. Définir **SCM** sur **Git**, coller l'**URL** du dépôt et fournir les identifiants **GitHub** (en utilisant le **Personal Access Token**) ;
8. Vérifier que le pipeline est connecté à la branche principale.
9. **Enregistrer et exécuter** : cliquer sur **Enregistrer**, puis sur **Build Now** pour tester la configuration du pipeline. Chaque **étape** peut être suivie en temps réel.



Vous avez maintenant configuré avec succès un pipeline **Jenkins** et l'avez connecté à **GitHub** à l'aide d'un **Jeton d'accès personnel**. Avec cette configuration, votre pipeline peut automatiquement récupérer le code, le construire, exécuter des tests et déployer des mises à jour à chaque modification poussée — créant ainsi un **workflow CI/CD** entièrement automatisé.

Automatiser les Processus de Compilation et de Test

Dans cette tâche, vous allez configurer un pipeline **Jenkins** pour automatiser les processus de build et de test. Vous relierez **Jenkins** à un dépôt **GitHub**, définirez un **Jenkinsfile** pour écrire le script du pipeline, et déclencherez un processus de build automatisé afin de vérifier son exécution.

Créer un Jenkinsfile

1. Saisir **Jenkinsfile** comme nom de fichier. Veiller à ne pas ajouter d'extension de fichier ;
2. Copier et coller le script de pipeline déclaratif suivant dans l'éditeur.

```
pipeline {
    agent any // run on any available agent

    stages {
        stage('Build') { // build stage
            steps {
                sh 'echo Building Project'
            }
        }

        stage('Test') { // test stage
            steps {
                sh 'echo Running Tests'
            }
        }

        stage('Deploy') { // deploy stage
            steps {
                sh 'echo Deploying Application'
            }
        }
    }
}
```

Ce pipeline déclaratif **Jenkins** s'exécute sur n'importe quel agent et suit une séquence simple **Build** → **Test** → **Deploy**. Chaque étape sert de point d'ancrage pour les commandes réelles : construction du projet et génération des artefacts, exécution des tests pour garantir la qualité, puis déploiement dans l'environnement cible. Sa structure simple et modulaire facilite son adaptation à une automatisation complète **CI/CD** dans **Jenkins**.

Résolution des problèmes courants

Introduction à Docker

Imaginez que vous souhaitez envoyer un projet à un ami, et qu'il fonctionne parfaitement sur votre ordinateur. Vous envoyez les fichiers, mais lorsque votre ami essaie de l'exécuter — rien ne fonctionne. Son système est différent, certains programmes requis ne sont pas installés, ou la configuration n'est pas la même. C'est là qu'intervient **Docker**.



Définition

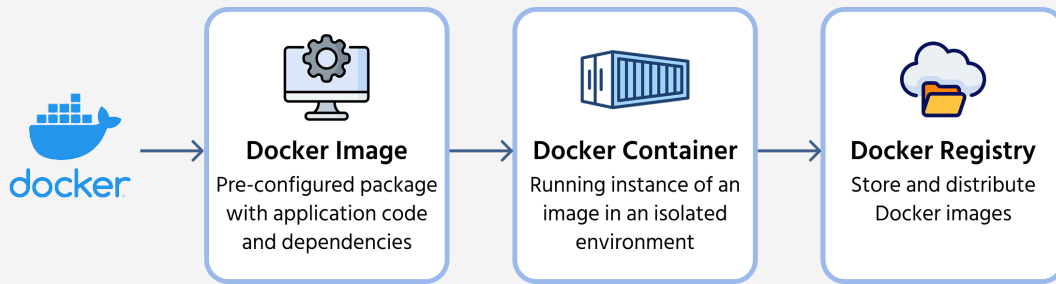
Docker est une plateforme gratuite et open source qui permet d'emballer votre application avec tout ce dont elle a besoin — comme des bibliothèques logicielles et des paramètres — dans un **conteneur**.

Considérez un conteneur comme une boîte contenant un jouet qui possède déjà des piles et une notice. Où que vous l'ouvriez, le jouet fonctionne exactement comme il le devrait.

Concepts clés de Docker

Image	→
Conteneur	→
Registre	→

Tous ces **concepts** sont essentiels pour le déploiement réussi de votre application. Ils fonctionnent ensemble dans l'ordre suivant, comme illustré dans le schéma ci-dessous :



Docker simplifie le déploiement des logiciels en utilisant des images, des conteneurs et des registres. Une **image Docker** regroupe une application avec toutes ses dépendances afin de garantir la cohérence entre les environnements. Lorsqu'elle est exécutée, l'image devient un conteneur, un environnement d'exécution isolé et léger. Les registres **Docker** comme **Docker Hub** permettent aux développeurs de stocker, partager et accéder à ces images, facilitant ainsi une livraison d'applications rapide, évolutive et portable.

Pourquoi utiliser Docker en DevOps ?

Docker est un outil essentiel dans les **pipelines CI/CD** car il rend les constructions et déploiements logiciels plus fiables. Il permet aux développeurs de créer le même environnement de développement sur différents ordinateurs, ce qui réduit les problèmes liés aux différences de systèmes.

En regroupant l'application avec tout ce dont elle a besoin, **Docker** facilite le passage fluide du logiciel du développement aux tests puis à la production. **Docker** s'intègre parfaitement aux **pipelines CI/CD**, permettant des tests et des déploiements automatisés.

En résumé, **Docker** aide à garantir que les applications fonctionnent de la même manière dans les environnements de développement, de test et de production.

Création d'une Application pour la Conteneurisation

Vous devez maintenant créer une application simple que vous exécuterez plus tard dans un conteneur.

Mais avant de pouvoir écrire du code, il vous faut un outil permettant d'exécuter du **JavaScript** sur votre ordinateur, d'installer les bibliothèques nécessaires et de lancer un serveur pour les tests.

Cet outil est **Node.js** — il servira de base à votre application et fournira tout ce dont vous avez besoin pour continuer.

Installation et configuration de Node.js

Rendez-vous sur le [🔗 site officiel de Node.js](https://nodejs.org/fr) et téléchargez la version **LTS (Long-Term Support)** — c'est l'option la plus stable et la plus largement prise en charge.

L'installateur comprend :

- **Node.js** — l'environnement d'exécution **JavaScript** ;
- **npm (Node Package Manager)** — l'outil pour installer et gérer les dépendances du projet.

Une fois l'installation terminée, vérifiez-la en exécutant ces commandes dans votre terminal ou invite de commandes :

```
node -v
```



Cela affichera la version de **Node.js** installée.

```
npm -v
```



Cela affichera la **version de npm** installée. Si les deux commandes renvoient des versions, l'installation a réussi.

Création d'un projet

Ensuite, créer un nouveau dossier de projet et s'y déplacer :

```
mkdir my-app && cd my-app
```



Initialisez maintenant le projet avec les paramètres par défaut. Cela générera un fichier `package.json` qui conserve les détails et les dépendances de votre projet :

```
npm init -y
```



Installez **Express**, un framework populaire qui facilite et accélère la création de serveurs :

```
npm install express
```



Création du fichier serveur

Le point d'entrée principal de l'application sera `server.js`. Ce fichier crée et configure le serveur **Express** et définit la manière dont il répond aux requêtes.

Créez un fichier nommé `server.js` dans le dossier de votre projet et ajoutez le code suivant :

```
// Import Express
const express = require('express');

// Create an Express application
const app = express();

// Set the port number using an environment variable or default to 3000
const PORT = process.env.PORT || 3000;

// Define a route for the homepage
app.get('/', (req, res) => {
  res.send('Hello from Dockerized App!');
});

// Start the server on the specified port
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

 Télécharger le fichier

Ce code importe **Express**, crée un serveur et définit la route `/` qui envoie un message à l'utilisateur. Le serveur démarre sur le port spécifié par la variable d'environnement **PORT**, ou utilise par défaut **3000** si la variable n'est pas définie.

Emballage de l'Application avec Docker

Vous disposez déjà d'une application **Node.js** fonctionnelle, et il est maintenant temps de l'emballer avec **Docker**. Alors que votre application gère les requêtes et les réponses, **Docker** garantit son exécution fiable dans n'importe quel environnement—sur votre machine, celle d'un collègue, ou dans le cloud.

Considérez votre application comme le moteur du projet, et **Docker** comme le conteneur qui transporte tout ce dont elle a besoin pour fonctionner : votre code, les dépendances et l'environnement d'exécution **Node.js**.

Création du Dockerfile

Le **Dockerfile** indique à **Docker** comment construire une image pour votre application **Node.js**. En termes simples, il s'agit d'une recette : des instructions étape par étape pour configurer l'environnement, installer les dépendances et exécuter l'application dans un conteneur.

Vous devez créer un **Dockerfile** dans votre projet **my-app**. Assurez-vous que le **Dockerfile** n'a pas d'extension de fichier, car cela pourrait provoquer des erreurs par la suite. Vous pouvez télécharger le fichier ci-dessous.

Ajoutez le contenu suivant :

```
# Use `Node.js 18` Alpine image
FROM node:18-alpine

# Set the working directory inside the container
WORKDIR /app

# Copy `package.json` and `package-lock.json`
COPY package*.json ./

# Install dependencies
RUN npm install

# Copy the rest of the application files
COPY . .

# Expose port 3000
EXPOSE 3000

# Command to run the application
CMD ["node", "server.js"]
```

 Télécharger le Dockerfile

FROM node:18-alpine



WORKDIR /app



COPY package*.json ./



RUN npm install



COPY . .



EXPOSE 3000



CMD ["node", "server.js"]



Remarque

Un **Dockerfile** est un modèle pour construire des images **Docker**. Chaque ligne représente une étape distincte, et Docker les exécute dans l'ordre afin de produire une image d'application prête à l'emploi.

Construction et exécution du conteneur Docker

Avant d'exécuter des commandes, assurez-vous que votre terminal se trouve dans le répertoire racine de votre projet, par exemple : `C:\Users\YourUsername\my-app`.

Vous devez maintenant construire une image **Docker** pour notre application afin qu'elle puisse s'exécuter dans un conteneur.

```
docker build -t my-app .
```



Cette commande construit une image **Docker** à partir du **Dockerfile** dans le dossier courant et la marque avec le tag **my-app**.

Ensuite, il est nécessaire d'exécuter le conteneur et de le rendre disponible sur le **port 3000** afin d'y accéder via un navigateur.

```
docker run -p 3000:3000 my-app
```



L'option **-p 3000:3000** associe le port **3000** à l'intérieur du conteneur au port **3000** de la machine locale. Cela rend l'application accessible depuis le navigateur.

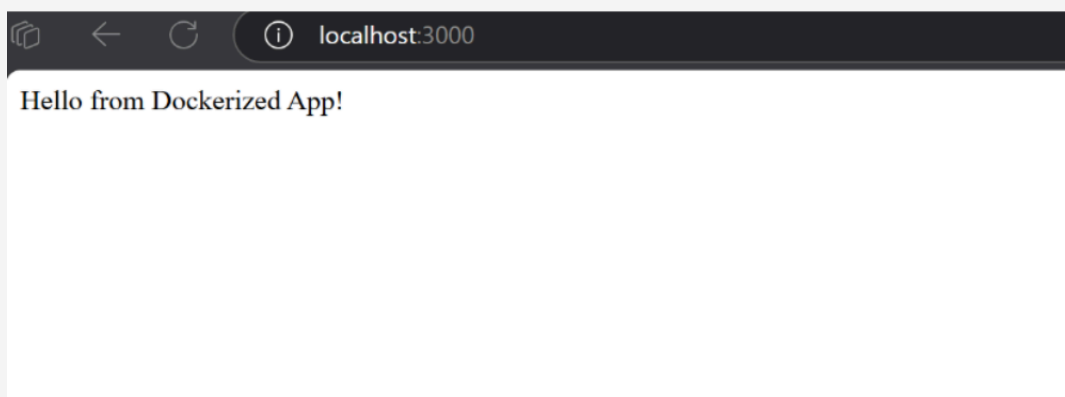
Ouvrir un navigateur et accéder à l'adresse suivante :

```
http://localhost:3000
```



Ici, **localhost** fait référence à votre propre ordinateur, et **3000** est le port sur lequel l'application écoute.

Si tout est correctement configuré, vous devriez voir :



Cela confirme que le serveur **Node.js** fonctionne à l'intérieur du conteneur **Docker**. L'application est désormais portable et s'exécutera de la même manière sur tout système disposant de **Docker** installé.

Introduction à Kubernetes

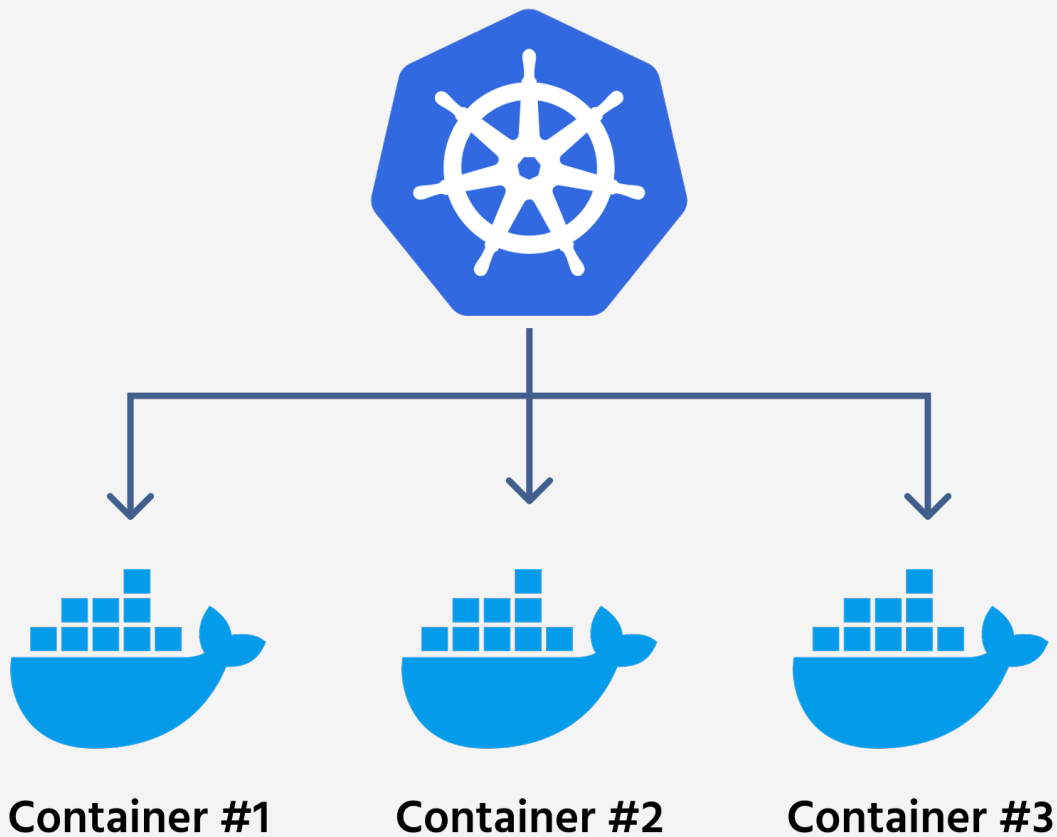
Maintenant que vous maîtrisez **Docker** et savez comment **containeriser des applications**, il est temps de passer à l'étape suivante : apprendre à **gérer des applications containerisées à grande échelle**, afin qu'elles puissent **s'adapter automatiquement, se remettre des pannes et fonctionner de manière fiable** dans n'importe quel environnement.

Qu'est-ce que Kubernetes ?



Définition

Kubernetes (K8s) est une plateforme de gestion des applications containerisées. En termes simples, elle permet d'exécuter et de contrôler plusieurs conteneurs simultanément.



Imaginez une **application web** composée de plusieurs services : **frontend**, **backend** et une **base de données**. Chaque service s'exécute dans un **conteneur Docker** distinct. À mesure que le nombre d'utilisateurs augmente, il devient nécessaire de lancer plusieurs instances du **backend** pour gérer la montée en charge. Disposer de copies supplémentaires permet de répartir les requêtes entre les conteneurs, réduisant ainsi la charge sur chacun et améliorant les temps de réponse.

C'est ici qu'intervient **Kubernetes** : il crée automatiquement de nouveaux conteneurs lorsque cela est nécessaire, répartit la charge et garantit la fiabilité de chaque service.

En résumé, **Kubernetes** transforme votre ensemble de conteneurs en une application intelligente et autonome, capable de **s'adapter à la charge** et de **se remettre des pannes**.

Pourquoi utiliser Kubernetes ?



Kubernetes offre plusieurs avantages majeurs, en particulier lorsque vos applications prennent de l'ampleur :

Mise à l'échelle automatique



Auto-réparation



Utilisation efficace des ressources



Cohérence de l'environnement



Configuration et Lancement du Déploiement Kubernetes

Maintenant que votre application fonctionne parfaitement dans des conteneurs, le prochain défi consiste à orchestrer et gérer ces conteneurs à grande échelle. C'est ici qu'intervient **Kubernetes**.

Pour commencer, vous allez définir l'état souhaité de votre application à l'aide de fichiers **YAML**. Un fichier décrira un **Deployment**, qui gère les mises à jour et la montée en charge, tandis qu'un autre définira un **Service**, qui expose votre application au réseau et la rend accessible aux utilisateurs.



Approfondir

YAML (YAML Ain't Markup Language) est un format lisible par l'humain pour les fichiers de configuration. Il est souvent utilisé pour décrire l'infrastructure, automatiser des processus et stocker les paramètres d'application.

Un **Deployment** garantit que le nombre souhaité d'instances de l'application (**Pods**) est toujours en cours d'exécution et facilite le déploiement de mises à jour ou le retour à des versions précédentes. Un **Service** expose ces conteneurs au réseau et répartit le trafic entrant entre eux.

Exécution de Minikube



Définition

Minikube est un outil léger permettant d'exécuter un **cluster Kubernetes** localement sur votre ordinateur. Il crée un environnement virtuel pour tester et explorer Kubernetes sans recourir à des services cloud.

Après avoir installé **Minikube** et l'avoir ajouté au PATH de votre système, ouvrez PowerShell (ou le terminal de votre choix) et exécutez :

```
minikube version
```



Cette commande vérifie si Minikube est disponible et affiche la version installée.

Une fois que vous avez confirmé que **Minikube** est installé, démarrez votre cluster **Kubernetes** local avec :

```
minikube start --memory=4096 --cpus=2 --driver=docker
```



Cette commande crée et lance un cluster **Kubernetes** local sur votre machine. Elle télécharge les composants nécessaires et démarre le plan de contrôle ainsi que les nœuds de travail.

Déploiement : Mise en production de l'application

Commencez par créer le fichier `deployment.yaml`. Un **Deployment** dans **Kubernetes** garantit qu'un nombre spécifique de **Pods** (instances de l'application) sont toujours en cours d'exécution et peuvent être mis à jour ou restaurés sans interruption de service.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app      # Deployment name
spec:
  replicas: 2        # Number of Pods to run
  selector:
    matchLabels:
      app: my-app    # Matches Pods with this label
  template:
    metadata:
      labels:
        app: my-app  # Label assigned to each Pod
    spec:
      containers:
        - name: my-app
          image: c1lowen/my-app  # Docker image to run
          ports:
            - containerPort: 3000  # Port inside the container
```

Dans ce manifeste, le paramètre `replicas` spécifie combien d'instances de **Pod** doivent fonctionner simultanément — dans ce cas, deux. Pour savoir quels **Pods** gérer, le **Deployment** utilise une combinaison de `selector` et de `labels` : le `selector` agit comme un filtre pour trouver les **Pods** avec un label spécifique, et le modèle de **Pod** définit ce label. Lorsque les labels correspondent, le **Deployment** considère ces **Pods** comme les siens et s'occupe de maintenir leur nombre, de les mettre à jour et de les redémarrer si nécessaire.

Tous les **Pods** fonctionnent sur le **port 3000**. Ils peuvent tous utiliser le même port car chaque **Pod** opère dans son propre espace réseau isolé, il n'y a donc aucun conflit entre eux.

Service : Accès réseau à l'application

Une fois le **Deployment** en place, un **Service** est nécessaire pour fournir un accès réseau et l'équilibrage de charge entre les **Pods**. Créer le fichier `service.yaml` :

```
apiVersion: v1
kind: Service
metadata:
  name: my-app-service    # Service name
spec:
  selector:
    app: my-app           # Connects to Pods with this label
  ports:
    - protocol: TCP
      port: 80             # Port exposed to the outside world
      targetPort: 3000     # Port inside the Pods
  type: LoadBalancer     # Provides external access and load
balancing
```

Ce manifeste définit un **Service** nommé `my-app-service` qui dirige le trafic vers les **Pods** portant le label `app: my-app`. Il écoute sur le port 80 en externe et redirige les requêtes vers le port 3000 à l'intérieur des **Pods**.

Le type `LoadBalancer` est important car il expose l'application à l'extérieur et répartit le trafic entre les **Pods**.

Déploiement de l'application sur Kubernetes

Une fois les fichiers **YAML** prêts, ils peuvent être appliqués au cluster :

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
```

La commande `kubectl apply -f` crée ou met à jour les ressources dans le cluster en fonction du contenu du fichier **YAML**.

Vérification de l'état

Après avoir appliqué les fichiers de configuration, il est important de s'assurer que tout a été déployé comme prévu. Étant donné que les **Pods** et les **Services** mettent un certain temps à démarrer, commencez par lister les **Pods** en cours d'exécution :

```
kubectl get pods
```



Si le fichier `deployment.yaml` spécifie `replicas: 2`, deux **Pods** devraient apparaître dans le résultat.

Pour vérifier que le **Service** fonctionne et est accessible :

```
kubectl get svc
```



Cela affichera l'adresse IP du **Service** ainsi que les ports par lesquels l'application est accessible.

Obtenir l'URL du service

Pour accéder à votre application, exécuter la commande suivante dans **PowerShell** ou votre terminal :

```
minikube service my-app-service --url
```



Cette commande demande à **Minikube** de retourner l'**URL** où votre application est exposée. **Minikube** vérifie le **Service** que vous avez créé (`my-app-service`) et fournit un lien que vous pouvez ouvrir dans votre navigateur.

Mise à l'échelle de l'application

Kubernetes facilite l'ajustement du nombre de **Pods** en cours d'exécution sans arrêter l'application.

Par exemple, pour passer de 2 à 4 réplicas :

```
kubectl scale deployment my-app --replicas=4
```



L'exécution de `kubectl get pods` affichera désormais quatre **Pods** actifs.

Kubernetes offre des outils puissants pour gérer les applications conteneurisées à n'importe quelle échelle. Un **Deployment** garantit que le nombre correct de Pods est en cours d'exécution et peut être mis à jour ou restauré facilement. Un **Service** expose l'application au réseau et équilibre le trafic entrant. Les fichiers de configuration **YAML** rendent la configuration claire et lisible, tandis que les commandes `kubectl` permettent d'appliquer et de gérer ces configurations en temps réel.